

raft-kv

A Distributed Key-Value Store with Raft Consensus,
Deterministic Simulation, and a Linearizability Checker

Brief: the argument, the architecture, and the evidence

1 The one-paragraph version

What this is

A key-value database that keeps identical copies of its data on several machines, so that any one of them can die without losing data or serving a wrong answer. Keeping machines in exact agreement while they crash and their network breaks is the problem called **consensus**, and this project solves it with the **Raft** algorithm, written from scratch in Python with no consensus libraries and zero runtime dependencies. But the consensus code is the smaller half of the project. The larger half is the evidence: a **deterministic simulator** that replays thousands of randomized disasters against the real consensus code as a pure function of a single integer, and a from-scratch **linearizability checker** that verifies the answers clients actually received could have come from a single, honest, non-distributed database.

1.1 The thesis

The project exists to make an argument, and the argument is the interesting part:

Consensus code is easy to write and nearly impossible to trust from example-based tests, because the bugs live in message interleavings you will never hit on a laptop loopback. So the system should be testable as a deterministic function of a seed.

This is the position taken by the FoundationDB database, and it drives every structural decision here. If every source of nondeterminism (time, message delay, loss, duplication, partition timing, crash timing, client behaviour, election jitter) is drawn from one seeded generator, then a whole 12-second cluster lifetime under sustained chaos becomes a **pure function of one integer**. That buys three things at once:

- thousands of universes run in seconds, because no real time passes;
- a failure report is one integer, not a flaky-test shrug;
- the failing schedule replays exactly, forever, under a debugger.

The whole architecture is bent around one precondition for this to be worth anything: **the simulated system must be the real system**. A simulator that tests a copy of the consensus logic tests nothing.

2 Three ideas you need, from first principles

Jargon: Consensus

Getting a group of unreliable machines to agree on one **ordered list of decisions**, such that once a decision is made it is permanent, and no two machines ever disagree about what decision number 7 was. Crucially, the group does not agree on “the value of key city”. It agrees on an ordered log of commands. If every machine holds the same log and runs the same deterministic program over it, every machine ends up with identical data, necessarily. Agreement on the data is derived free from agreement on the order. This is called a **replicated state machine**.

Jargon: Quorum, and why it makes split brain impossible

Every important decision requires a strict majority of the cluster. The reason this works is a fact about sets, not computers: **any two majorities of the same set must share a member**, because $2 + 2 > 3$. So a network partition can leave at most *one* side with a majority; the other side is mathematically unable to act. Without this, two halves of a split cluster would each keep accepting writes and diverge permanently, which is called **split brain**. In this code the rule is one line: `len(self.config) // 2 + 1`.

Jargon: Linearizability

The correctness target, and the strongest single-object guarantee. A system is linearizable if every operation appears to take effect **instantaneously at some single moment between the client sending the request and receiving the response**, consistent with real time. The practical meaning: if your write finished at 10:00:01, every read starting at 10:00:02, on any node, must see it. The distributed system becomes indistinguishable from a single-threaded program holding one copy of the data. It is a property of the *observable history*, not of the implementation, which is exactly why it can be checked from the outside without trusting the code.

3 The architecture, and the decision it turns on

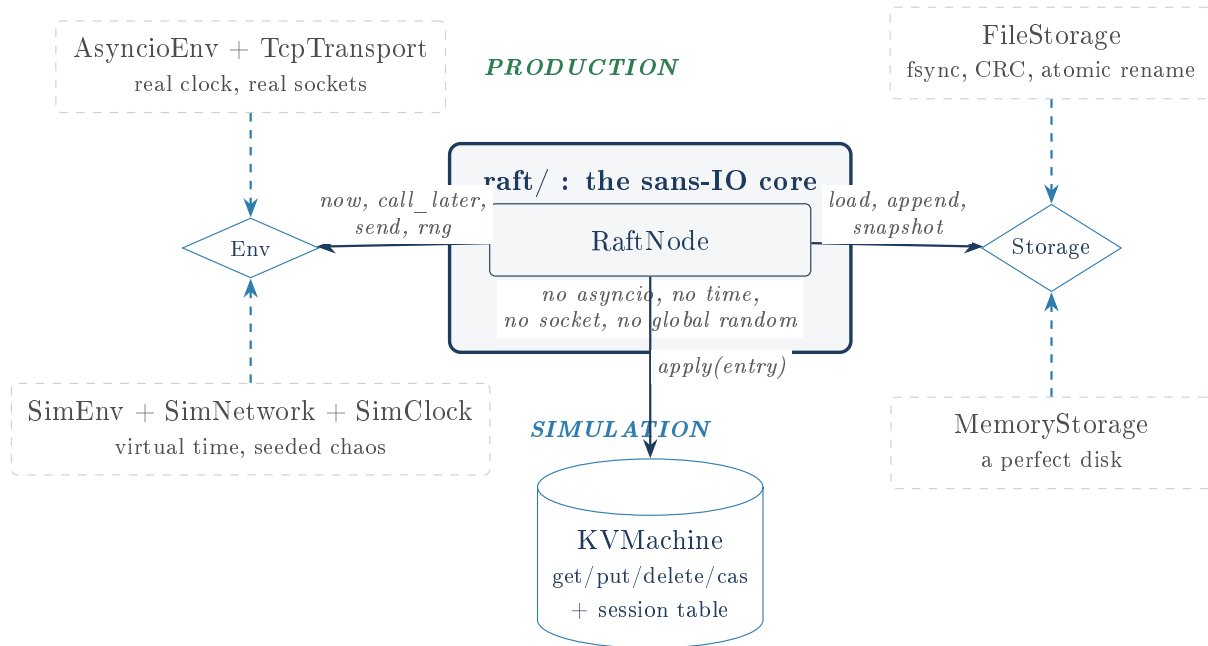


Figure 1: The whole system in one picture. **RaftNode** can touch the world only through two narrow protocols. Swap what is plugged into them and the identical consensus code runs over real TCP or inside the simulator.

Sans-IO is the whole ballgame

Sans-IO (“without IO”) means the protocol implementation performs no input or output itself: it never opens a socket, never reads a clock, never sleeps. **RaftNode** reacts to exactly three stimuli (a message arrived, a timer fired, the local application asked for something) and emits effects through exactly two protocols, **Env** (`now`, `call_later`, `send`, `rng`) and **Storage**. The payoff is the entire project. Because the node takes its randomness from `env.rng` rather than the global `random` module, and its time from `env.now()` rather than `time.time()`, the simulator can hand it a seeded generator and a virtual clock and it cannot tell. So “run 5000 chaos schedules in a few minutes” becomes an ordinary function call, and **every bug the simulator finds is a production bug**, because it is literally the production code. There is no test double of the algorithm anywhere in this repository, and **raft/** contains no `if testing:` branch.

The module layout follows directly, with the dependency arrow pointing inward:

Module	Role
raft/	the consensus core: <code>RaftNode</code> (576 lines), the log, storage, messages, and the <code>Env</code> protocol. Imports nothing outward.
kv/	the replicated state machine: get, put, delete, cas, plus the session table
transport/	real asyncio TCP, length-prefixed JSON frames, and <code>AsyncioEnv</code>
sim/	virtual clock, seeded chaos network, the harness, the batch runner
checker/	history recording and the linearizability search
server.py, client.py, cli/	one TCP node, a retrying client, the <code>raftkv</code> command

4 Raft, in one page

Every node is a follower, a candidate, or a leader. Time is divided into **terms**, an ever-increasing logical clock; at most one leader exists per term, and any message carrying a higher term instantly demotes its receiver to follower.

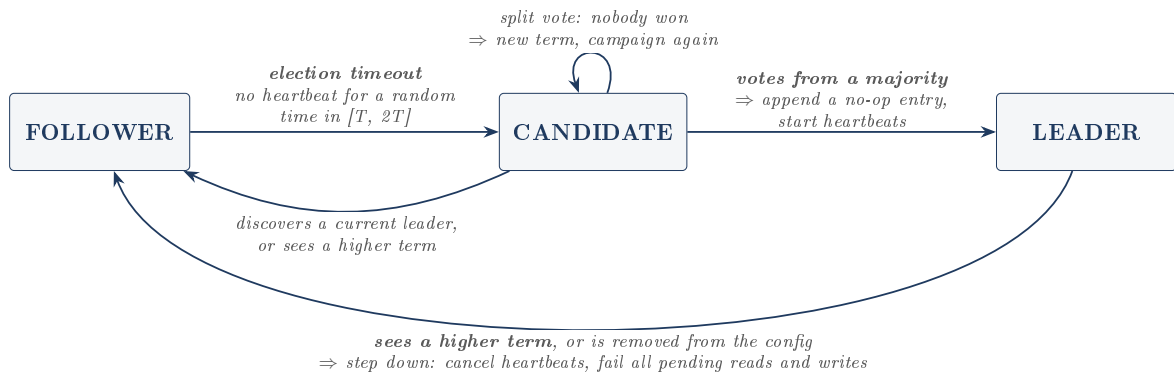


Figure 2: The role state machine. Note the asymmetry: there is no follower-to-leader edge and no leader-to-candidate edge. Power is only taken through an election and only surrendered by stepping down.

Election. Followers wait a *randomized* time (uniform in $[T, 2T]$) without hearing a heartbeat, then increment the term, vote for themselves, and ask for votes. Randomization is what prevents everyone from campaigning at once and splitting the vote forever. Two rules carry the weight: the vote is **persisted with fsync before the reply is sent** (a node that forgot its vote after a crash could vote twice in one term, producing two leaders), and voters **refuse any candidate whose log is less up to date than their own**. That second rule is what makes committed data survive every leader change automatically: a winner needs a majority, any majority overlaps the majority holding a committed entry, and those voters will refuse anyone missing it.

Replication. The leader appends a command to its own log, fsyncs it, and broadcasts an `AppendEntries` message. That message carries the index and term of the entry *immediately before* the new ones. If a follower disagrees there, it rejects and returns a **conflict hint**, the first index of the conflicting term, so the leader can back up in one jump rather than one entry per round trip. Agreement on that one preceding entry proves agreement on the entire prefix, by induction over the fact that indexes are never reused and one leader exists per term. Once a majority stores an entry, it is **committed**: permanent, present in every future leader’s log.

The subtlest rule in Raft, and the reason for the no-op

A leader may **not** commit an entry from an earlier term merely because it is now replicated on a majority. If it did, a different node whose log is more up to date on other grounds could still win a later election, not have that entry, and overwrite it: an entry reported committed would vanish. So the code checks `self.log.term_at(candidate) == self.current_term` before advancing the commit index (the paper's section 5.4.2). To make progress on the inherited backlog anyway, a fresh leader immediately appends a **noop** entry of its own term; when that commits by the ordinary majority rule, everything before it commits transitively and safely. The no-op is not a nicety: without it a leader receiving no writes could never commit its predecessor's backlog at all.

Crash safety falls out of one discipline: never acknowledge before `fsync`, and make every on-disk structure either atomic-rename recoverable or scan-and-truncate recoverable. `hardstate.json` (term and vote) is written temp-file, `fsync`, rename, then **the directory is fsynced too**, which is the step most implementations forget. `log.bin` is append-only records of [4-byte length][4-byte CRC32][JSON]; on recovery the file is scanned and the first short or bad-CRC record marks a torn tail from a crash mid-write, which is truncated away. Discarding data is provably safe here precisely because nothing is acknowledged before `fsync` returns, so a half-written record was never promised to anyone.

Snapshots replace an applied prefix of the log with an image of the state machine (KV data and the session table), sent to hopelessly lagging followers via `InstallSnapshot`. **Membership changes** use the dissertation's single-server approach: add or remove exactly one node per config entry, so any two configurations share a common majority and disjoint quorums cannot form, which is why no two-phase joint-consensus protocol is needed.

5 Linearizable reads: ReadIndex

A leader cannot know that it is still the leader

This is the trap that makes reads harder than writes. Suppose `n1` is partitioned away from `n2` and `n3`, which elect a new leader and accept writes. `n1` has heard nothing, so it still believes it leads, and its local data looks perfectly fine *to it*. It will happily serve a value that is minutes stale. Nothing in its own state betrays it. Leadership is not a local fact, so reading from local memory is unsafe.

The fix is **ReadIndex**: before answering a read, the leader must *prove* its currency. It records `readIndex = commitIndex`, broadcasts a heartbeat round carrying a sequence number, and waits for acks from a majority for that round; only then does it read local state. A deposed leader in a minority partition is not merely unlikely to answer, it is **structurally unable** to, because those acks cannot cross the partition. When it eventually sees the higher term it steps down, and stepping down explicitly fails every pending read, so the client is told `not_leader` and retries elsewhere.

There is a second precondition, and the README credits the chaos harness with finding it: a fresh leader must first commit an entry **of its own term** before serving reads, because its `commit_index` is a stale local variable that may be lower than what is actually committed cluster-wide. Reads arriving too early are parked with no read index and assigned one the instant the leader's no-op commits.

The alternative that was refused, and why the reason matters

Leader leases let a leader serve reads for `election_timeout - clock_drift` after one heartbeat quorum, saving a round trip per read. This project refuses them, and documents the refusal: leases buy latency at the price of a safety argument that depends on **bounded clock drift between physical machines**. In a project whose entire thesis is testable correctness, importing an untestable assumption about hardware clocks is a bad trade. ReadIndex costs one heartbeat round and assumes nothing about clocks.

6 Exactly-once writes

Why retries corrupt data, and why the fix must live in the replicated state

A client sends a cas. The leader commits it, then dies before replying. The client times out and retries against the new leader, which has no idea this is a retry and applies it again. For a cas the retry now *fails* (the value already changed), so the client is told its operation failed when in fact it succeeded. The fix: every write carries (`client_id`, `seq`), and the state machine remembers per client the last seq applied and **the result it returned**; a duplicate returns the cached result without re-executing.

The part that is easy to get wrong is where the table lives. It must be **inside the replicated state**, going through the log and riding inside snapshots. A leader-local cache would silently downgrade the system from exactly-once to at-least-once precisely at failover, which is exactly when it matters. Exactly-once is a state machine feature, not a networking feature.

7 The evidence machine

7.1 One seeded run

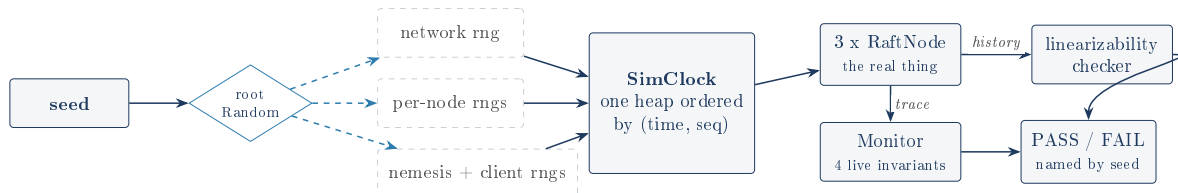


Figure 3: Everything derives from the seed; everything funnels through one ordered event heap; two independent oracles judge the result. The tie-break on a sequence number inside that heap is load-bearing: without it, two events at the same virtual instant would run in an undefined order and determinism would die.

Each seed runs 12 virtual seconds of a 3-node cluster with 3 concurrent clients on 3 keys, under 5% message loss, 5% duplication, randomized delay, and a nemesis firing at Poisson intervals: random bisecting partitions, crash with delayed restart, election-timer clock skew (0.5x to 1.8x), and heals. Because time is a variable rather than something you wait for, those 12 virtual seconds cost roughly 0.1 real seconds. There is no `sleep` anywhere in the simulator.

Chaos stops at 70% of the run and the cluster is then healed and stabilized so in-flight operations can drain. That is not cheating, it is what makes the check bite: **incomplete operations are the easiest thing for a linearizability checker to accept**, because it can always declare they never happened. A run that ended mid-chaos would look clean while testing very little.

Four invariants are checked live, each failure naming its seed: election safety (at most one leader per term, tracked across the whole run), leader completeness (a new leader's log is checked

against everything ever applied, at the moment it wins), state machine safety (two nodes applying different entries at one index is an immediate failure), and log matching.

7.2 The linearizability checker

Written from scratch: a Wing & Gong exhaustive search with Lowe’s memoization. Repeatedly pick an operation that is *minimal* (real-time order permits it to go next), apply it to a model of a single register, and check the model reproduces what the client actually observed; backtrack when it cannot. Two decompositions make it tractable: keys are independent registers so each key is checked separately, and the set of linearized operations is a **bitmask integer**, making the memo key a single machine word. Incomplete operations branch both ways, implemented elegantly as a weaker acceptance test (every *completed* op must be linearized; leftovers may be declared never to have happened).

Three outcomes, never two

OK, VIOLATION, and UNKNOWN. The third is the honest one. Linearizability checking is NP-complete in general, so any checker must eventually refuse; this one refuses explicitly above 63 operations per key or 2,000,000 explored states. The property that matters: it **never returns a false PASS or a false FAIL**, only sometimes a shrug. Note also that exhausting the search space with no explanation found is a *proof* of violation, not a timeout.

7.3 Testing the tests

A safety net that never fires is indistinguishable from one that cannot fire

This is the best idea in the suite, and it appears three times independently. `test_broken_raft_is_caught` monkeypatches the quorum rule from `// 2 + 1` to `// 2`, deliberately making split brain possible, and asserts the harness detects a violation within 40 seeds. The checker is fed known-bad synthetic histories (stale read, lost update, phantom value) and must reject them. And `test_same_seed_same_run` runs a seed twice and compares a full history fingerprint, so determinism is asserted structurally rather than assumed.

The README tells one story on itself that is worth repeating, because it is the same lesson at a smaller scale. The first “checker budget” test used 40 incomplete concurrent puts and passed, but it passed for the wrong reason: a history of purely incomplete operations is trivially linearizable by declaring none of them happened, so the search finished instantly and exercised none of the mechanism it claimed to test. The fixed test buries a real violation under 20 *completed* concurrent puts, forcing the exhaustive search the budget exists to bound. A test that passes tells you nothing until you know why.

8 Verified, not quoted

The claims below were **executed on this machine** for this document, from a clean virtual environment (Linux, Python 3.12.3, 8 cores), rather than copied from the README.

Claim (README)	Observed	Verdict
57 tests, all passing	57 passed in 51.41s	confirmed
5000 seeds swept	5000 seeds	confirmed
564170 client ops total	564170 client ops total	confirmed, exactly
0 failures	0 failure(s)	confirmed
84.7s (59.0 seeds/s), 10 workers	257.0s (19.5 seeds/s), 8 workers	differs; see below

```

1 $ .venv/bin/raftkv sim --seeds 5000 --workers 8
2 5000 seeds in 257.0s (19.5 seeds/s), 564170 client ops total, 0 failure(s)

```

Listing 1: The headline sweep, run for this document

The op count reproduced to the digit, and that is the real result

The README’s run used 10 workers; this one used 8, on a different machine, on a different day. The operation count came back **identical**. That is not a coincidence and it is not a weak check: it is the determinism claim confirmed end to end. Had any part of the system consulted a wall clock, a hash seed, a set iteration order, or the global `random` module, the same 5000 seeds would have produced a different number. They did not.

The timing did not reproduce, and the gap is larger than the worker count explains

84.7s at 10 workers versus 257.0s at 8 workers is a factor of about 3; the worker ratio accounts for at most 1.25. This is not a discrepancy in the code: throughput is deliberately *outside* the seed and depends on the machine and its load. **This run shared the machine with other substantial workloads**, which is the most likely explanation, but that is an **inference**: it was not isolated or measured, and the original run was not repeated under matched conditions. The honest position: the timing figure is a machine-and-day observation rather than a property of the project, while the two figures that *are* properties of the project, the op count and the failure count, both reproduced exactly.

What the headline number does and does not cover

“5,000 randomly generated disasters and 564,170 reads/writes with 0 contradictions” is true and was reproduced. Its scope is narrower than it sounds, and saying so is what makes it defensible:

- **5000 seeds is a deep search of one fixed scenario**, not a broad search across scenarios: every seed runs the same recipe (3 nodes, 12 virtual seconds, the same fault probabilities) and varies only timing, targets and ordering.
- **The ops are get, put and cas**. The simulated client has no `delete` branch, so `delete` is never exercised end to end by any of the 5000 seeds.
- **The simulated disks are perfect**. The simulator uses `MemoryStorage`, an ideal disk that never tears a write. The excellent torn-tail CRC recovery code in `FileStorage` is therefore never touched by the sweep; it is covered only by targeted unit tests. This is the largest gap between what the simulator tests and what production runs, and the README names it.
- **Membership changes are never exercised by the sweep** either: the nemesis crashes, partitions and skews, but never adds or removes a server.
- **“0 contradictions” means no violation fired**, not that every history was proven linearizable. `SimResult.ok` accepts `UNKNOWN`, and the batch runner never counts it, so the sweep cannot distinguish 5000 proofs from 4990 proofs and 10 shrugs. The configured width almost certainly keeps every history inside the budget, but the runner does not *show* that, and it easily could.

9 Honest assessment

9.1 What is genuinely strong

- **The sans-IO boundary is real, not aspirational.** `raft/` imports `enum`, `typing` and its own siblings, and nothing else. The central architectural claim survives a `grep`.
- **The subtle rules are all present and correct:** vote persistence before reply, the up-to-date check, the section 5.4.2 own-term commit restriction with its no-op, a conflict hint that can only move `next_index` downward (so a confused hint costs a round trip, never correctness), the directory `fsync` after rename, config effective on append, and a leader that defers its own removal until it commits.
- **The tests test the tests**, three independent ways.
- **The honesty is load-bearing:** the UNKNOWN outcome, the documented rejection of leader leases with reasons, and the README's own limits section are the work of someone optimizing for being right rather than for looking right.

9.2 Weaknesses, ranked by what would hurt most under questioning

1. **The simulator's disk is perfect, so the crash-recovery code is the least-tested important code in the project.** The project's thesis is "search the schedule space"; storage faults are simply outside the searched space. This is the answer to "what would you do next" and should be volunteered, not conceded.
2. UNKNOWN is **invisible** in `SimResult.ok` and in the batch runner's output. A few lines would make the headline strictly stronger.
3. **Replication is naive and worst-case quadratic.** Every heartbeat to a lagging follower re-sends the *entire* remaining suffix, because `next_index` only advances on a reply. Correct, since `AppendEntries` is idempotent, but with a 30 ms heartbeat and a follower 10,000 entries behind that is the whole suffix serialized to JSON every 30 ms until the first reply lands. There is no batch-size cap.
4. `_load_snapshot()` **re-reads and re-parses the entire persistent state, including the whole log, on every `InstallSnapshot` send**, then ships the snapshot whole with no chunking.
5. **No pre-vote**, so a healed node forces one pointless re-election through term inflation.
6. **Sessions never expire**, and the table is inside the replicated state and every snapshot. Since the CLI builds a fresh client with a fresh random id per invocation, every `raftkv put` from the command line permanently adds a row.
7. **Scale:** Python, JSON on the wire, one `fsync` per append batch, one TCP connection per client request. The README's first honest limit says plainly that this is a correctness project and not a fast store, which is the right call.

9.3 Smaller things a careful reader will find

- The checker validates `get` and `cas` results against the model but treats `put` and `delete` as pure state transitions, so `delete`'s `existed` flag is never checked.
- The log-matching invariant breaks at the first index where terms agree, inferring the prefix from the Log Matching Property, which is the property under test. It is partially circular; other layers still catch what it misses.
- `next_index` initialization disagrees with itself across three functions (off by one in `_change_config`); self-correcting, so the tests never see it.
- `_config_from`'s index filter is effectively vacuous at both call sites.

10 What to take away

Five sentences

One. Consensus reduces to keeping an ordered log identical across machines; Raft does it by electing one leader per term who alone appends, and by requiring a majority for every decision, so that the overlap between any two majorities makes contradiction impossible.

Two. Everything rests on sans-IO: `RaftNode` touches the world only through `Env` and `Storage`, so identical code runs over TCP and under the simulator, and every simulator bug is by construction a production bug.

Three. Because all nondeterminism funnels through one seeded generator and one ordered event heap, a chaotic cluster lifetime is a pure function of one integer: thousands of disasters searchable in minutes, every failure reproducible forever.

Four. Correctness is judged from the outside by a from-scratch linearizability checker that proves, refutes, or honestly says `UNKNOWN`, and never guesses.

Five. The safety nets are themselves tested, because a net that never fires is indistinguishable from one that cannot.

Likely question	Honest answer
Why not use an existing Raft library?	The point was to learn where the subtlety lives. Section 5.4.2 and the <code>ReadIndex</code> term-commit precondition are two places a plausible shortcut is a real linearizability bug, and the chaos harness found the second one during development.
Is 5000 seeds meaningful?	It is a deep search of one fixed scenario, not a broad one. It reproduced to the digit at a different worker count, which is the determinism claim confirmed.
What is not tested?	Storage faults (the simulated disk is perfect), <code>delete</code> end to end, membership changes under chaos, and anything beyond single-digit node counts on localhost.
What would you do first?	Inject torn writes and <code>fsync</code> reordering into simulated storage, so the sweep covers the recovery paths that are currently only unit-tested. Then report <code>UNKNOWN</code> counts.
Why <code>ReadIndex</code> , not leases?	Leases trade a round trip for an assumption about bounded clock drift between machines. In a project about testable correctness, an untestable hardware assumption is a bad trade.

Every figure in the verification section was produced by running the code on this machine, not by quoting the README. Anything inferred is labelled inferred. Where code and documentation disagreed, the code was taken as the truth. For the exhaustive treatment, including the full state machine, the election and replication paths, the partition analysis, and a module-by-module walkthrough, see the companion Deep Dive.